ToricGT IV: Oracle Trajectory Classifiers for Behavior, Reasoning Quality, and Bits-per-byte Control

A NeurIPS-style fourth-iteration research article for ToricGT heads, oracle transformers, trajectory typing, and model-behavior training

Amelie Schreiber
June 2026

Abstract

ToricGT I made toric geometry operational for graph-token reasoning by turning cones, normal fans, affine semigroups, BGG/Koszul certificates, Gale duality, toric ideals, and GraphCG behavior corridors into finite train-time supervision. ToricGT II made the object dynamical: a reasoning step became a standardized packet whose filtered simplicial complex, multigraded persistence module, derived object, sheaf, vector bundle, differential-form features, and toric chart codes evolve under repeated learned graph-to-graph updates. ToricGT III moved the same framework outside the model into model-context protocols, markdown memory, knowledge graphs, graph-structured vector databases, projected retrieval heads, quantized toric context codes, and long 1–10M-token iterative context windows driven by tropical ring attention.

ToricGT IV adds an oracle layer. The goal is to train either an additional classifier head on top of an existing reasoning model or a full oracle transformer that classifies completed and partial reasoning trajectories into structured behavioral, epistemic, computational, and stylistic types: accuracy, faithfulness, evidence grounding, tool discipline, compliance, safety, refusal appropriateness, tone, personality corridor, verbosity, proof rigor, uncertainty calibration, efficiency, retrieval hygiene, memory hygiene, and graph-of-thought compositionality. The oracle is not a post hoc sentiment classifier. It is a mathematically constrained trajectory-type model whose outputs live in a property poset, a Stanley-Reisner compatibility complex, a toric property fan, a multigraded label module, and a sheaf of local-window judgments. It can be trained as a lightweight head, a frozen-backbone probe, a low-rank adapter stack, or a full transformer over graph-structured trajectory packets.

We develop a rigorous theory of oracle trajectory classification. We define trajectory objects, property lattices, behavior sheaves, toric property polytopes, persistence-label modules, derived residuals, and calibrated multi-label distributions. We prove equivariance, proper-scoring, side-information, isotonic projection, sheaf-gluing, persistence-stability, toric-margin, oracle-guided policy-improvement, and ergodic-occupancy theorems. We then give a concrete training paradigm: label acquisition, weak supervision, verifier distillation, human/rubric calibration, contrastive trajectory pairs, bits-per-byte-gated promotion, oracle-assisted reinforcement or preference training, GFlowNet reward shaping, retrieval filtering, memory pruning, MCP tool-call gating, and test-time behavior projection. The guiding principle is unchanged: the oracle exists to improve model behavior and reasoning, not to decorate the system with attractive mathematics. A trajectory label is useful only when it predicts future bytes, improves answer quality, reduces error, improves controllability, or prevents a costly failure under auditable deployment constraints.

Design contract. ToricGT IV treats behavior as a property of reasoning trajectories, not merely final logits. The oracle reads graph-of-thought packets, MCP/memory/KG context events, filtered complexes, certificate signatures, verifier traces, and output drafts. It emits calibrated property distributions and certified incompatibility warnings. Those signals are used to improve training only through score-safe, ablated channels: supervised loss weighting, rejection sampling, preference optimization, reward shaping, retrieval gating, memory hygiene, or lightweight steering. Anything that improves the oracle but worsens held-out bits-per-byte, verified task quality, or artifact efficiency remains teacher-side.

Contents

1 Purpose and continuity with ToricGT I–III

1.1 The fourth problem

The first three ToricGT papers progressively enlarge the object of supervision. ToricGT I supervises graph-token reasoning with finite toric, tropical, BGG, Koszul, Gale-dual, and behavior-corridor certificates. ToricGT II studies iterative reasoning dynamics by attaching filtered simplicial complexes, multigraded persistence modules, sheaves, vector bundles, differential forms, derived objects, and ergodic summaries to each reasoning step. ToricGT III embeds the same machinery in the external memory substrate: model-context protocols, markdown memory, knowledge graphs, projected graph-structured vector databases, quantized retrieval codes, and 1–10M-token long-context tropical ring attention.

ToricGT IV asks a different but necessary question. Suppose the model can construct rich reasoning trajectories. Which trajectories are good? Which are accurate, efficient, compliant, well calibrated, grounded, concise, rigorous, or stable in tone? Which ones are unsafe, hallucinated, overconfident, sycophantic, evasive, too verbose, too terse, tool-abusive, memory-leaky, or brittle? A base language-model loss sees only bytes. A verifier sees only a task slice. A safety classifier often sees only a final answer. A useful reasoning agent needs a
emphtrajectory oracle: a trainable model that classifies the entire trajectory, including hidden graph-of-thought structure, context assembly, memory retrieval, tool use, proof obligations, verifier traces, and final response properties.

The ToricGT IV thesis is:

> A trajectory classifier becomes useful for training reasoning agents when its labels are structured enough to be compositional, local-to-global, calibrated, compatible with graph symmetries, and predictive of downstream log score, reasoning quality, or behavior reliability.

The classifier may be a small additional head on an existing ToricGT backbone, or a full oracle transformer trained on trajectory packets. The head is cheaper and easier to ablate; the full oracle is more expressive and can be used as a teacher, dataset judge, and training-time reward model.

1.2 Relation to ToricGT I

ToricGT I introduced a graph-token model family

$$F_\theta = D \circ \Phi_L \circ \cdots \circ \Phi_1 \circ T, \tag{1.1}$$

where T tokenizes typed graphs, the blocks Φ_ℓ are permutation-equivariant over node and edge tokens, and D decodes graph, node, edge, or byte-level outputs. Its toric layer is a finite-certification layer: tropical active faces, Newton-polytope cells, toric ideals, Cox degrees, BGG differentials, Gale-dual labels, and Euler-Koszul certificates organize hidden states without forcing a deployed byte model to carry a full symbolic algebra system.

ToricGT IV uses the same finite-certification layer as an oracle input. A classifier that sees only final text cannot distinguish a correct answer obtained from a stable proof from a lucky answer obtained by hallucinated intermediate reasoning. A trajectory oracle can classify the proof skeleton, hidden certificate residuals, evidence path, and behavior-corridor occupancy. It therefore supplies a richer target for improving the base model.

1.3 Relation to ToricGT II

ToricGT II replaced isolated certificates by dynamical objects. A reasoning step x_t induces a filtered complex $\mathcal{K}_t$, a persistence module $\mathcal{M}_t$, a derived object $\mathcal{D}_t$, a sheaf/vector-bundle object $\mathcal{F}_t$, differential-form features ω_t, and a toric chart code c_t. Repeated application of a learned graph-to-graph map

produces

$$(x_0, \mathcal{K}_0, \mathcal{M}_0, \mathcal{D}_0, \mathcal{F}_0, c_0) \to (x_1, \mathcal{K}_1, \mathcal{M}_1, \mathcal{D}_1, \mathcal{F}_1, c_1) \to \cdots . \tag{1.2}$$

ToricGT IV classifies this sequence. Instead of using Betti drift, sheaf gluing, or ergodic occupation only as auxiliary metrics, we treat them as features and sometimes labels. A trajectory type can include "stable persistence", "late evidence collapse", "refusal boundary crossing", "proof branch merged coherently", or "tone drift after memory retrieval".

1.4 Relation to ToricGT III

ToricGT III externalized the trajectory into the context substrate. Model-context-protocol calls, markdown memory blocks, knowledge-graph neighborhoods, tool/resource/prompt records, and long-context windows become typed graph objects. ToricGT IV adds an oracle that reads those context events and predicts properties of the resulting reasoning trajectory. For example, the oracle may decide that a tool-call plan is accurate but inefficient, that retrieved memory is relevant but stale, that a citation block is grounded but too verbose, or that a long-context fill policy is likely to exceed the marginal utility threshold.

The official model-context-protocol specification treats MCP as a protocol connecting LLM applications to external data sources and tools, with server features such as tools, resources, and prompts. ToricGT III treats those protocol events as graph-structured retrieval objects; ToricGT IV treats them as oracle-classification evidence for tool discipline, permission safety, grounding, memory hygiene, and answer reliability [5].

1.5 Contributions

1. We formalize a trajectory property space containing accuracy, tone, personality, compliance, safety, efficiency, evidence grounding, reasoning quality, tool discipline, retrieval hygiene, memory hygiene, and output-style properties.
2. We define two oracle architectures: a low-cost trajectory-classifier head on the base model and a full oracle transformer over graph-structured trajectory packets.
3. We impose mathematical structure on labels: property posets, Stanley-Reisner compatibility complexes, toric property polytopes, multigraded label modules, Cech-style local-to-global gluing, persistence stability, and derived residual certificates.
4. We prove core theorems that justify using oracle outputs to improve training: equivariance, proper scoring, side-information bounds for bits-per-byte, isotonic projection, sheaf gluing, stability under persistence perturbations, margin-preserving quantization, oracle-guided policy improvement, and ergodic control.
5. We give a concrete training paradigm: dataset construction, label sourcing, verifier distillation, human calibration, weak supervision, frozen-probe training, adapter training, full-oracle training, oracle distillation into the base model, and deployment gating.
6. We specify how oracle labels improve the base model through supervised fine-tuning, preference optimization, GFlowNet rewards, trajectory rejection, retrieval and memory filtering, context-window planning, MCP tool-call gating, and low-rank behavior projection.

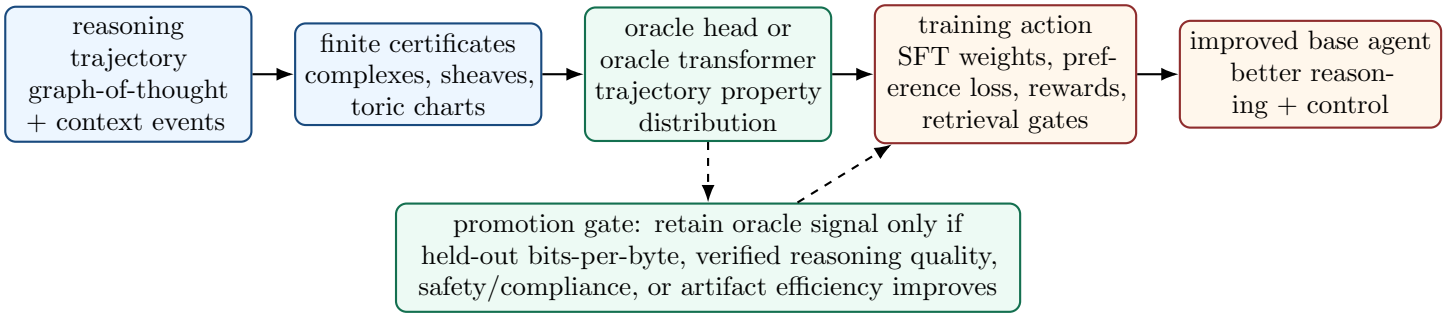

Figure 1: ToricGT IV system view. The oracle reads complete or partial trajectory packets plus finite ToricGT certificates, emits calibrated property distributions, and influences training only through auditable channels.

2 The trajectory object and its property space

2.1 Trajectory packets

Definition 2.1 (ToricGT trajectory packet)**.** *Fix a horizon T, padding sizes, and finite type vocabularies. A ToricGT trajectory packet is*

$$\tau = (G_\tau, X_{0:T}, C_{0:T}, R_{0:T}, V_{0:T}, Y, B_{<T}), \tag{2.1}$$

where $G_\tau = (V_\tau, E_\tau)$ is a directed acyclic graph-of-thought topology; X_t is the standardized reasoning-step tensor at vertex or time t; C_t contains certificate objects; R_t contains retrieval/memory/MCP event summaries; V_t contains verifier, evidence, and tool-return fields; Y is the final response or intermediate output; and $B_{<T}$ is the strict byte prefix available before scoring the relevant output bytes.

The packet is deliberately redundant. A classifier head may use only hidden states and a few summaries. A full oracle transformer may use the whole packet, including graph edges, certificate matrices, Cech overlaps, retrieved knowledge-graph paths, and long-context blocks. Redundancy helps with auditability: if the oracle says "low grounding", one should be able to inspect whether the cause is missing evidence, stale memory, unsupported tool output, or a failed gluing condition.

Definition 2.2 (Certificate field)**.** *A certificate field attached to a trajectory packet is a tuple*

$$C_t = (\mathcal{K}_t, \mathcal{M}_t, \mathcal{D}_t, \mathcal{F}_t, \omega_t, \Sigma_t, \beta_t, \gamma_t), \tag{2.2}$$

where $\mathcal{K}_t$ is a filtered simplicial complex, $\mathcal{M}_t$ is a multigraded persistence module, $\mathcal{D}_t$ is a bounded derived object or a finite chain complex representative, $\mathcal{F}_t$ is a sheaf/vector-bundle object over a local cover, ω_t is a differential-form feature, Σ_t is a toric fan or occupied empirical fan, β_t is a Betti or barcode summary, and γ_t stores Cox degrees, one-dimensional-cone activations, tableaux/crystal labels, or root-system chamber codes.

2.2 Atomic properties

A trajectory oracle should classify more than "good" or "bad". The property set must be broad enough to support targeted improvement and narrow enough to be learnable.

Definition 2.3 (Atomic trajectory property)**.** *An atomic trajectory property is a tuple*

$$p = (\text{name}, \text{scope}, \text{polarity}, \text{scale}, \text{evidence\_rule}, \text{action\_rule}). \tag{2.3}$$

The scope is one of token, step, branch, tool call, memory retrieval, proof obligation, final answer, or whole trajectory. The polarity indicates desirable, undesirable, neutral, or context-dependent status. The scale is binary, ordinal, continuous, distributional, or set-valued. The evidence rule states what finite information can support the label. The action rule states how the training system may use the label.

Core property families include:

1. **Epistemic properties:** factual accuracy, mathematical correctness, uncertainty calibration, citation adequacy, evidence grounding, verifier agreement, contradiction risk.
2. **Reasoning properties:** decomposition quality, proof rigor, branch usefulness, merge consistency, persistence stability, derived residual, algebraic certificate validity, root-system/tableau normal-form correctness.
3. **Efficiency properties:** context economy, tool-call economy, retrieval precision, token budget discipline, latency estimate, bits-per-byte utility, artifact-byte utility.
4. **Behavior properties:** compliance with user intent, safety boundary handling, refusal appropriateness, safe-alternative quality, tone, personality corridor, verbosity, empathy, confidence, domain register.
5. **Memory and MCP properties:** permission correctness, stale-memory risk, memory contamination, KG neighborhood relevance, tool schema compliance, resource provenance, markdown section hygiene.

2.3 Property posets and compatibility complexes

Many properties are ordered. "Fully grounded" implies "partially grounded"; "severe hallucination" implies "unsupported claim"; "unsafe compliance" is incompatible with "safe refusal" but may coexist with "polite tone". We encode this explicitly.

Definition 2.4 (Property poset)**.** *Let $\mathcal{P}$ be a finite set of atomic properties. A property poset is a partial order $(\mathcal{P}, \preceq)$ where $p \preceq q$ means that property q entails property p. A label vector $y \in \{0,1\}^{\mathcal{P}}$ is upward-consistent if $y_q = 1$ and $p \preceq q$ imply $y_p = 1$.*

Definition 2.5 (Compatibility complex)**.** *A compatibility complex is an abstract simplicial complex $\Delta \subseteq 2^{\mathcal{P}}$. A set $S \subseteq \mathcal{P}$ is admissible if $S \in \Delta$. Minimal nonfaces encode forbidden coactivations, such as high confidence with no evidence, unsafe tool use with compliance approval, or verbose style with low token budget when concise answer is required.*

The Stanley-Reisner ideal of Δ is

$$I_\Delta = \left\langle \prod_{p \in F} z_p : F \notin \Delta \right\rangle \subset k[z_p : p \in \mathcal{P}]. \tag{2.4}$$

For soft labels $\hat{y}_p \in [0,1]$, the differentiable nonface penalty is

$$\mathcal{L}_{\mathrm{SR}}(\hat{y}) = \sum_{F \in \mathrm{MinNonFaces}(\Delta)} w_F \prod_{p \in F} \hat{y}_p. \tag{2.5}$$

This is a compact algebraic way to prevent incompatible behavior labels from being simultaneously high.

2.4 Toric property polytopes

Definition 2.6 (Toric property polytope)**.** *Choose an embedding $a_p \in M$ for every atomic property $p \in \mathcal{P}$. The property polytope is*

$$P_{\mathcal{P}} = \mathrm{Conv}\{a_p : p \in \mathcal{P}\} \subset M_{\mathbb{R}}. \tag{2.6}$$

Given oracle logits $\ell_p(\tau)$, the tropical property score is

$$\psi(\tau) = \max_{p \in \mathcal{P}} \{\ell_p(\tau)\}. \tag{2.7}$$

The active property face is the set of properties whose logits attain or nearly attain the maximum. Its normal-fan cone is the local behavior type of the trajectory.

The purpose of the toric polytope is not decorative. It gives an audit for classifier decisions. If two properties share an edge in $P_{\mathcal{P}}$, they are expected to exchange under small perturbations; if they are separated by many facets, a sudden swap suggests instability. One-dimensional cones of the normal fan become interpretable axes: evidence support, refusal strength, proof rigor, warmth, verbosity, tool reliance, and uncertainty.

Definition 2.7 (Property fan margin)**.** *Let $r^\star = \arg\max_p \ell_p(\tau)$ be unique. The unnormalized property fan margin is*

$$\Delta_{\mathcal{P}}(\tau) = \min_{q \neq r^\star} \big(\ell_{r^\star}(\tau) - \ell_q(\tau)\big). \tag{2.8}$$

For grouped or multi-label properties, replace the singleton $r^\star$ by the active face and take the minimum gap to inactive competitors.

Large margins are useful because they make oracle labels robust to quantization, adapter perturbations, and context-window compression.

3 Oracle architectures

3.1 Additional oracle head

The cheapest implementation is a classifier head on top of an existing ToricGT or transformer backbone. Let $H_\theta(\tau)$ denote hidden trajectory features: pooled graph-token states, final hidden states, certificate summaries, retrieved evidence features, and output-draft features. An oracle head is

$$O_\phi(\tau) = \big(q_\phi(y_p = 1 \mid \tau)\big)_{p \in \mathcal{P}}, \tag{3.1}$$

with optional ordinal, continuous, or distributional outputs.

A lightweight head should be used first because it tests whether the base representation already contains property information. If a frozen head cannot learn useful trajectory labels, the base model likely lacks the needed internal structure or the labels are too noisy. If a frozen head performs well, then training the base model with the oracle signal is safer.

3.2 Full oracle transformer

A full oracle transformer is a separate model Ω_ψ that reads trajectory packets directly. Its input tokens include:

1. step tokens: hidden-state summaries, output text chunks, verifier scores, evidence vectors;
2. graph tokens: branch/merge edges, tool-call edges, retrieval edges, memory-update edges;
3. certificate tokens: boundary matrices, Betti tables, barcode summaries, Cox degrees, one-dimensional-cone occupancies, tableaux, root-system chamber labels;
4. context tokens: MCP events, markdown blocks, KG triples, source identifiers, permissions, freshness, and provenance;
5. response tokens: final answer draft, citations, refusal text, or structured output.

The full oracle can use ordinary attention, tropical ring attention, projected retrieval into the ToricGT III graph-structured vector database, and local certificate computation.

Definition 3.1 (Oracle transformer)**.** *An oracle transformer is a map*

$$\Omega_\psi : \mathcal{T} \to \prod_{p \in \mathcal{P}} \mathcal{Y}_p \tag{3.2}$$

from the space $\mathcal{T}$ of bounded trajectory packets to property-output spaces $\mathcal{Y}_p$. It is graph-invariant if relabeling graph-of-thought vertices, KG nodes, memory nodes, or equivalent MCP event ids leaves whole-trajectory labels unchanged, and graph-equivariant if local labels are correspondingly relabeled.

3.3 Head versus full oracle

The head and full oracle have different uses.

1. **Frozen head:** diagnostic and cheap; identifies properties linearly or shallowly encoded in hidden states.
2. **Adapter head:** trains upper-layer adapters to make behavior properties separable without rewriting the base model.
3. **Full oracle:** most accurate teacher; expensive; suitable for offline labeling, trajectory filtering, reward shaping, and distillation.
4. **Distilled micro-oracle:** low-rank or quantized classifier retained only if it improves deployment metrics.

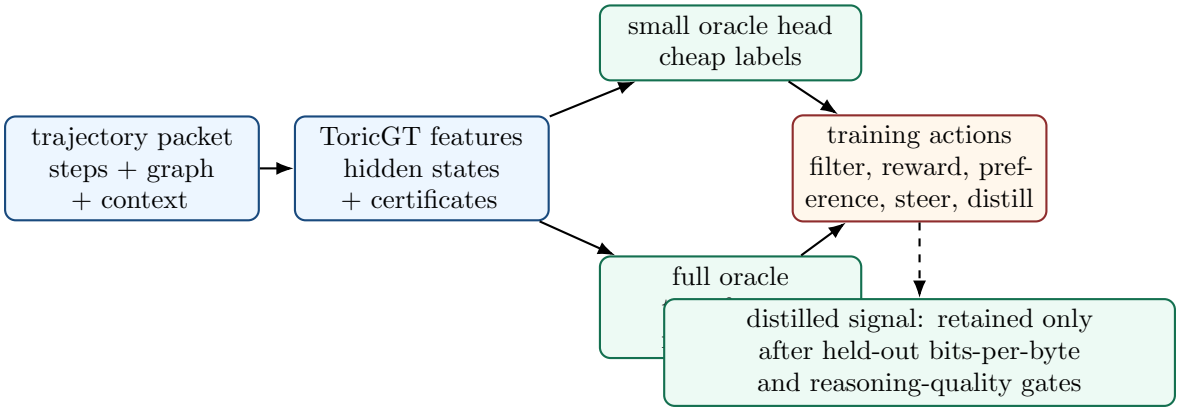

Figure 2: Two oracle routes. A small head tests whether property labels are already encoded; a full oracle transformer supplies stronger offline supervision and reward signals.

4 Mathematical foundations

4.1 Equivariance and invariance

Trajectory labels should not depend on arbitrary graph ordering. Renumbering graph-of-thought vertices, MCP resource handles, or KG entity ids should relabel local judgments and preserve global judgments.

Theorem 4.1 (Oracle equivariance)**.** *Let Π be a group of admissible relabelings of trajectory packet tokens. Suppose the trajectory tokenizer is equivariant, all attention masks are conjugated by token relabeling, all projections are shared over token types, and graph-level pooling is symmetric. Then the oracle transformer is equivariant for local labels and invariant for global labels.*

Proof. The proof is by composition. Equivariant tokenization maps a relabeled packet to a permuted token table. Shared linear projections commute with row permutations. Masked softmax attention and tropical attention commute with simultaneous row/column permutation of their score matrices. Shared feed-forward layers, residuals, and layer normalizations are row-wise and therefore commute with token permutation. By induction over layers, hidden token states relabel equivariantly. Local decoders applied row-wise inherit equivariance. Global pooling by sum, mean, log-sum-exp, max, or invariant attention is symmetric, hence invariant. □

4.2 Proper scoring and calibration

Theorem 4.2 (Cross-entropy identifies conditional property probabilities)**.** *Let $Y_p \in \{0,1\}$ be a binary property and let Z be the oracle input. Over all measurable predictors $q(Z) \in (0,1)$, the expected binary cross-entropy*

$$\mathbb{E}\big[-Y_p \log q(Z) - (1-Y_p)\log(1-q(Z))\big] \tag{4.1}$$

is uniquely minimized almost surely by $q^\star(Z) = \mathbb{P}(Y_p = 1 \mid Z)$.

Proof. Condition on $Z = z$ and write $\eta = \mathbb{P}(Y_p = 1 \mid Z = z)$. The conditional risk is

$$R(q) = -\eta \log q - (1-\eta)\log(1-q). \tag{4.2}$$

Differentiating gives $R'(q) = -\eta/q + (1-\eta)/(1-q)$, which vanishes exactly at $q = \eta$. The second derivative is positive on $(0,1)$, so this is the unique minimizer. □

This theorem matters because an oracle trained by proper scoring can be used as a calibrated control signal. A label "unsafe tool path with probability 0.83" is more useful than a hard rejection without uncertainty.

4.3 Bits-per-byte side-information theorem

The oracle improves compression only if its signal predicts future bytes or helps the base model choose better trajectories. The basic information identity is the correct discipline.

Theorem 4.3 (Oracle side information and bits-per-byte)**.** *Let B be the next byte or byte block, C the strict prefix context, and Z an oracle-visible trajectory property signal that is also prefix-measurable. Let $q_0(b \mid C)$ be the optimal predictor using C and $q_1(b \mid C, Z)$ the optimal predictor using (C, Z). Then the expected reduction in base-two log loss is*

$$\mathbb{E}\big[-\log_2 q_0(B \mid C) + \log_2 q_1(B \mid C, Z)\big] = I(B; Z \mid C). \tag{4.3}$$

For byte block length L, the expected bits-per-byte reduction is $I(B; Z \mid C)/L$ before artifact and compute costs.

Proof. The optimal log-loss predictor is the true conditional distribution. Thus the first expected loss is $H(B \mid C)$ and the second is $H(B \mid C, Z)$, both measured in bits. Their difference is $H(B \mid C) - H(B \mid C, Z) = I(B; Z \mid C)$. Dividing by L gives the byte-normalized value. □

Corollary 4.4 (Artifact-adjusted oracle utility)**.** *If storing or computing an oracle code costs A bits over an evaluation block of L bytes, then a necessary condition for net compression utility is*

$$I(B; Z \mid C) > A. \tag{4.4}$$

More generally, an oracle component should be retained only when held-out bits-per-byte utility plus predeclared quality/control gains clears the deployment threshold.

4.4 Property poset projection

Oracle labels must obey entailments. Raw logits need not.

Definition 4.5 (Isotonic property projection)**.** *Given soft scores $s \in \mathbb{R}^{\mathcal{P}}$ and a property poset $(\mathcal{P}, \preceq)$, the isotonic projection is*

$$\Pi_{\preceq}(s) = \arg\min_{z \in \mathbb{R}^{\mathcal{P}}} \sum_{p \in \mathcal{P}} (z_p - s_p)^2 \quad \textit{subject to} \quad p \preceq q \Rightarrow z_p \le z_q. \tag{4.5}$$

Theorem 4.6 (Nonexpansiveness of property projection)**.** *The isotonic projection $\Pi_{\preceq}$ is unique and nonexpansive in Euclidean norm:*

$$\|\Pi_{\preceq}(s) - \Pi_{\preceq}(s')\|_2 \le \|s - s'\|_2. \tag{4.6}$$

Proof. The feasible set is a closed convex cone cut out by linear inequalities $z_p - z_q \le 0$. Euclidean projection onto a nonempty closed convex set in a Hilbert space is unique and firmly nonexpansive. Applying that standard projection theorem gives the result. □

4.5 Sheaf of local trajectory judgments

A long trajectory is judged locally. Accuracy might fail in a citation block, tone might drift in a refusal branch, and tool discipline might fail at one MCP call. Local judgments must glue.

Definition 4.7 (Behavior sheaf on a trajectory cover)**.** *Let $\mathcal{U} = \{U_i\}$ be a cover of trajectory vertices, context blocks, or time intervals. For each open set U_i, let $\mathcal{F}(U_i)$ be a vector space of local property logits or calibrated probabilities. Restriction maps $\rho_{ij} : \mathcal{F}(U_i) \to \mathcal{F}(U_i \cap U_j)$ compare labels on overlaps. A family (s_i) is a global behavior section when $\rho_{ij} s_i = \rho_{ji} s_j$ on every overlap.*

Proposition 4.8 (Least-squares gluing bound)**.** *Suppose each $\mathcal{F}(U_i)$ is a finite-dimensional Hilbert space and restrictions are norm-nonincreasing. Let $\delta_{ij} = \|\rho_{ij} s_i - \rho_{ji} s_j\|$. The minimum Cech mismatch*

$$\mathcal{L}_{\text{glue}}(s) = \sum_{i<j} \|\rho_{ij} s_i - \rho_{ji} s_j\|^2 \tag{4.7}$$

vanishes if and only if the local sections glue to a global section. If the associated Cech Laplacian has positive smallest nonzero eigenvalue λ_1, then the distance from $s = (s_i)$ to the space of global sections is at most $\lambda_1^{-1/2} \mathcal{L}_{\text{glue}}(s)^{1/2}$.

Proof. The Cech coboundary d^0 maps local sections to overlap disagreements. Its kernel is exactly the space of compatible local sections, i.e. global sections of the sheaf on the cover. The mismatch is $\|d^0 s\|^2$. On the orthogonal complement of $\ker d^0$, the Cech Laplacian $(d^0)^* d^0$ has spectrum bounded below by λ_1, so $\|d^0 s\|^2 \ge \lambda_1 \operatorname{dist}(s, \ker d^0)^2$. □

4.6 Persistence stability for trajectory classification

Definition 4.9 (Persistence-aware oracle)**.** *An oracle is persistence-Lipschitz if there are constants L_H, L_D such that for trajectories τ, τ',*

$$\|O(\tau) - O(\tau')\| \le L_H \|H(\tau) - H(\tau')\| + L_D d_B(\mathrm{Dgm}(\mathcal{K}_\tau), \mathrm{Dgm}(\mathcal{K}_{\tau'})), \tag{4.8}$$

where d_B is bottleneck distance between persistence diagrams.

Proposition 4.10 (Robustness under filtration perturbation)**.** *If a trajectory oracle is persistence-Lipschitz and the filtration functions of two trajectory complexes differ by at most ε in sup norm, then*

$$\|O(\tau) - O(\tau')\| \le L_H \|H(\tau) - H(\tau')\| + L_D \varepsilon. \tag{4.9}$$

Proof. The standard stability theorem for persistent homology gives $d_B(\mathrm{Dgm}(\mathcal{K}_\tau), \mathrm{Dgm}(\mathcal{K}_{\tau'})) \le \varepsilon$ when filtration functions differ by at most ε. Substitute into the Lipschitz definition. □

4.7 Toric chamber stability and quantization

Theorem 4.11 (Margin-preserving property quantization)**.** *Let oracle logits be affine on a toric property chamber: $\ell_p(h) = \langle a_p, h\rangle + b_p$. Suppose quantized coefficients $(\hat{a}_p, \hat{b}_p)$ satisfy*

$$\max_p \sup_{h\in K} |\ell_p(h) - \hat{\ell}_p(h)| \le \varepsilon \tag{4.10}$$

for a compact hidden-state set K. If a trajectory $h \in K$ has property fan margin $\Delta_{\mathcal{P}}(h) > 2\varepsilon$, then quantization preserves the active property label.

Proof. Let $p^\star$ be the unique active label. For any $q \neq p^\star$,

$$\hat{\ell}_{p^\star}(h) - \hat{\ell}_q(h) \ge \ell_{p^\star}(h) - \ell_q(h) - 2\varepsilon \ge \Delta_{\mathcal{P}}(h) - 2\varepsilon > 0. \tag{4.11}$$

Thus $p^\star$ remains uniquely active. □

4.8 Oracle-guided policy improvement

The oracle can guide training by assigning reward to trajectories, but it must not overpower task correctness.

Definition 4.12 (Oracle-shaped reward)**.** *Let $R_{\text{task}}(\tau)$ be a verifier or human-preference reward and $u_\phi(\tau)$ an oracle desirability score derived from property probabilities. The shaped reward is*

$$R_\phi(\tau) = R_{\text{task}}(\tau) + \alpha u_\phi(\tau) - \beta c_{\text{artifact}}(\tau) - \gamma c_{\text{risk}}(\tau). \tag{4.12}$$

Theorem 4.13 (KL-regularized oracle improvement)**.** *Let $\pi_0(\tau \mid x)$ be a base trajectory policy and let R_ϕ be bounded. The unique maximizer of*

$$\mathbb{E}_{\tau\sim\pi}[R_\phi(\tau)] - \eta\, \mathbb{E}_x\, \mathrm{KL}(\pi(\cdot \mid x)\|\pi_0(\cdot \mid x)) \tag{4.13}$$

over distributions absolutely continuous with respect to π_0 is

$$\pi_\phi(\tau \mid x) = \frac{\pi_0(\tau \mid x)\exp(R_\phi(\tau)/\eta)}{Z_\phi(x)}. \tag{4.14}$$

The objective value is at least that of π_0.

Proof. For fixed x, the optimization is the Donsker-Varadhan variational formula. Completing the square in relative entropy gives

$$\mathbb{E}_\pi R_\phi - \eta\, \mathrm{KL}(\pi\|\pi_0) = \eta \log Z_\phi(x) - \eta\, \mathrm{KL}(\pi\|\pi_\phi), \tag{4.15}$$

which is maximized uniquely by π_ϕ. Choosing $\pi = \pi_0$ is feasible, so the maximum is at least the base value. □

4.9 Ergodic oracle occupancy

For deployed agents, we care about long-run behavior frequencies: how often the agent becomes overconfident, verbose, unsafe, or tool-inefficient.

Theorem 4.14 (Oracle occupancy under stationary reasoning dynamics)**.** *Let (X_t) be an ergodic Markov chain on compact trajectory-step space with invariant distribution μ. Let $g_p(X_t)$ be a bounded oracle score for property p. Then*

$$\frac{1}{T}\sum_{t=0}^{T-1} g_p(X_t) \to \int g_p(x)\, d\mu(x) \tag{4.16}$$

almost surely. The same holds for any bounded function of finitely many oracle properties, certificate residuals, and bits-per-byte-local losses.

Proof. This is the Markov-chain ergodic theorem for bounded measurable observables under ergodicity. The compactness assumption ensures no integrability issue for bounded oracle outputs. □

5 Label construction and supervision sources

5.1 Label taxonomy

ToricGT IV uses a typed label schema rather than a flat classifier. A minimal practical taxonomy is:

1. **Correctness:** correct, partially correct, unverifiable, contradiction, mathematical error, citation error, tool-output mismatch.
2. **Grounding:** grounded in retrieved evidence, unsupported but plausible, unsupported and false, memory-stale, KG-path unsupported, MCP-resource mismatch.
3. **Reasoning quality:** valid decomposition, invalid branch, coherent merge, circular proof, missing lemma, excessive search, unstable persistence, high derived residual.
4. **Behavior:** helpful, evasive, overconfident, sycophantic, emotionally mismatched, too terse, too verbose, domain-register mismatch.
5. **Compliance and safety:** follows user intent, asks needed clarification, refuses correctly, refuses unnecessarily, unsafe compliance, policy-boundary ambiguity.
6. **Efficiency:** good token economy, poor token economy, redundant retrieval, unnecessary tool call, insufficient tool use, high artifact cost, positive bits-per-byte utility.
7. **Memory/MCP discipline:** permission-respecting, stale-memory use, memory contamination, tool schema violation, resource provenance missing, prompt injection susceptibility.

5.2 Human labels, verifier labels, and weak labels

No single label source is sufficient. Human labels capture tone, personality, refusal appropriateness, and nuanced compliance. Verifiers capture task success. Programmatic audits capture algebraic and graph consistency. Weak labels capture scale. The oracle should explicitly model source reliability.

Definition 5.1 (Multi-source label model)**.** *Let Y_p be the latent true property and let $L_{p,s}$ be a label from source s for property p. A multi-source label model has source confusion matrices*

$$\Theta_{p,s}(a,b) = \mathbb{P}(L_{p,s} = b \mid Y_p = a), \tag{5.1}$$

with priors tied by property family. Training alternates between estimating posterior soft labels $\mathbb{P}(Y_p \mid L_{p,1:S}, \tau)$ and fitting the oracle to those soft labels.

5.3 Synthetic certificate labels

Many labels can be generated exactly. For toric algebra tasks, compute toric ideals, Cox degrees, Hilbert bases, Cech residuals, Betti tables, and small resolutions offline. For graph tasks, compute shortest-path consistency, cut certificates, matching feasibility, proof-tree validity, and branch-merge compatibility. For behavior tasks, synthetic labels should be used carefully: they are useful for nonface constraints and tool-schema compliance but insufficient for human tone.

5.4 Contrastive pairs

ToricGT IV benefits from pairs of trajectories that solve the same task but differ in one property. Examples:

1. correct proof versus proof with one false lemma;
2. grounded answer versus hallucinated citation;
3. concise technical answer versus verbose but correct answer;
4. safe refusal with alternative versus unhelpful blanket refusal;
5. efficient two-call tool plan versus redundant ten-call plan;
6. coherent branch merge versus inconsistent branch merge;
7. current memory retrieval versus stale memory retrieval.

Contrastive pairs train axes to be interpretable and reduce label superposition.

6 Training objectives

6.1 Core supervised oracle objective

Let $\hat{y}_p = O_\phi(\tau)_p$ be a probability, ordinal distribution, or continuous score. The core objective is

$$\mathcal{L}_{\text{sup}} = \sum_{p \in \mathcal{P}} w_p \ \text{CE}(y_p, \hat{y}_p), \tag{6.1}$$

with label-source reliability weights and missing-label masks.

For ordinal labels $y_p \in \{0, \dots, K_p\}$, use cumulative logits

$$\hat{F}_{p,k}(\tau) = \mathbb{P}(Y_p \le k \mid \tau) \tag{6.2}$$

with monotonicity penalties $\max(0, \hat{F}_{p,k} - \hat{F}_{p,k+1})$.

6.2 Structured oracle loss

The structured loss combines property-poset, compatibility, calibration, sheaf, toric, and certificate terms:

$$\mathcal{L}_{\text{oracle}} = \mathcal{L}_{\text{sup}} + \lambda_{\preceq}\mathcal{L}_{\preceq} + \lambda_{\text{SR}}\mathcal{L}_{\text{SR}} + \lambda_{\text{cal}}\mathcal{L}_{\text{cal}} + \lambda_{\text{glue}}\mathcal{L}_{\text{glue}} \tag{6.3}$$

$$+ \lambda_{\text{fan}}\mathcal{L}_{\text{fan}} + \lambda_{\text{pers}}\mathcal{L}_{\text{pers}} + \lambda_{\text{der}}\mathcal{L}_{\text{der}} + \lambda_{\text{ret}}\mathcal{L}_{\text{ret}} + \lambda_{\text{eff}}\mathcal{L}_{\text{eff}}. \tag{6.4}$$

The components are:

$$\mathcal{L}_{\preceq} = \sum_{p \preceq q} \max(0, \hat{y}_q - \hat{y}_p)^2, \tag{6.5}$$

$$\mathcal{L}_{\text{SR}} = \sum_{F \in \text{MinNonFaces}(\Delta)} w_F \prod_{p \in F} \hat{y}_p, \tag{6.6}$$

$$\mathcal{L}_{\text{cal}} = \sum_{b \in \text{bins}} \frac{|b|}{N} \left(\text{conf}(b) - \text{acc}(b)\right)^2, \tag{6.7}$$

$$\mathcal{L}_{\text{fan}} = \mathbb{E}_\tau \max(0, \gamma - \Delta_{\mathcal{P}}(\tau))^2, \tag{6.8}$$

$$\mathcal{L}_{\text{der}} = \sum_t \|d_{t+1} f_t - f_{t-1} d_t\|_F^2, \tag{6.9}$$

$$\mathcal{L}_{\text{eff}} = \max(0, c_{\text{artifact}} - \Delta\widehat{\text{bits-per-byte}} \cdot L)^2. \tag{6.10}$$

The loss is not always enabled. Use audit-only mode first; then probe-only; then adapter fine-tuning; then full-oracle teacher training; then base-model distillation.

6.3 Trajectory contrastive objective

For a pair (τ^+, τ^-) where τ^+ is preferred for property p, use

$$\mathcal{L}_{\text{rank}} = -\log \sigma \left(s_p(\tau^+) - s_p(\tau^-) - m_p \right), \tag{6.11}$$

where m_p is a margin. For multi-property pairs, the margin is weighted by a signed property-difference vector.

6.4 Oracle-to-base distillation

The base model should not depend on a large oracle at inference unless ablations justify the cost. Distillation trains a small student signal:

$$\mathcal{L}_{\text{distill}} = \mathrm{KL}\big(O_\psi(\tau) \,\|\, O_{\text{small}}(h_\theta(\tau))\big) + \lambda \|z_{\text{small}}\|_0^{\text{soft}}. \tag{6.12}$$

The sparsity term encourages a compact code: a few property logits, margins, and warning flags rather than a full explanatory trace.

7 Using the oracle to improve the model

7.1 Data selection and rejection sampling

The simplest use is dataset filtering. Generate multiple candidate trajectories for a problem. Score them with the oracle. Keep trajectories that are correct, grounded, efficient, and behavior-compatible. Reject trajectories with high hallucination probability, unsafe-compliance risk, stale memory, or low verifier agreement. This improves supervised fine-tuning data without changing inference.

7.2 Loss reweighting

For a training token block $b_{1:L}$ generated by trajectory τ, define a quality weight

$$w(\tau) = \exp\left(\alpha u_{\text{quality}}(\tau) - \beta u_{\text{risk}}(\tau) - \gamma u_{\text{cost}}(\tau)\right). \tag{7.1}$$

The weighted language-model loss is

$$\mathcal{L}_{\text{LM,w}} = \sum_t w(\tau_t) \big[-\log p_\theta(b_t \mid b_{<t}) \big]. \tag{7.2}$$

Weights must be clipped to avoid destroying base distributional coverage.

7.3 Preference optimization

For two candidate outputs or trajectories (τ_a, τ_b), define oracle preference

$$\Delta_\phi = u_\phi(\tau_a) - u_\phi(\tau_b). \tag{7.3}$$

Use it to construct preference pairs for DPO-style or reward-model training, but only after calibration checks show that the oracle agrees with held-out human/verifier judgments.

7.4 GFlowNet and graph-of-thought reward shaping

A graph-of-thought search policy can use terminal reward

$$R(\tau) = R_{\text{task}}(\tau) \exp\left(\alpha u_{\text{oracle}}(\tau) - \beta c_{\text{risk}}(\tau) - \gamma c_{\text{length}}(\tau)\right). \tag{7.4}$$

This biases exploration toward trajectories that solve the task by desirable reasoning rather than by accidental final-answer matching.

7.5 Retrieval and memory gates

For a memory candidate m and current trajectory prefix $\tau_{<t}$, retrieve by

$$S(q, m) = \langle Pq, Pm \rangle + \lambda_\Sigma \langle \Sigma(q), \Sigma(m) \rangle + \lambda_O \langle O(q), O(m) \rangle - \lambda_{\text{bad}} r_{\text{risk}}(m). \tag{7.5}$$

The oracle term prevents retrieval of semantically similar but behaviorally incompatible memories, such as stale code snippets, unsafe tool traces, or overly verbose prior answers.

7.6 MCP tool-call gating

For each proposed MCP tool call, classify the local tool trajectory: schema compliance, permission safety, relevance, expected utility, and prompt-injection risk. Let a be a proposed action. Execute only if

$$\mathbb{E}[U(a) \mid \tau] - \lambda_{\text{risk}} \mathbb{P}(\text{risk} \mid \tau, a) - \lambda_{\text{cost}} c(a) > \kappa. \tag{7.6}$$

This gives a principled link between the MCP substrate of ToricGT III and the behavior-classification objective of ToricGT IV.

7.7 Test-time steering

When the oracle is differentiable with respect to hidden states, it can define a local projection:

$$h^+ = \arg\min_{\tilde{h}} \frac{1}{2} \|\tilde{h} - h\|_G^2 + \lambda \, \text{dist}(O_\phi(\tilde{h}), K_{\text{desired}})^2 + \eta \, \mathcal{L}_{\text{SR}}(O_\phi(\tilde{h})). \tag{7.7}$$

This should be used sparingly. In many cases the safer method is to train the base model with oracle-shaped examples rather than to steer at inference.

8 Toric, sheaf, and representation-theoretic refinements

8.1 Cox-coordinate behavior codes

Let Σ be a behavior fan with one-dimensional cones $\rho \in \Sigma(1)$. A Cox-style behavior code uses variables z_ρ for primitive behavior axes. A trajectory's soft code is $z_\rho(\tau) \in [0, 1]$. The Cox degree records a behavior class:

$$\deg z^u = \sum_\rho u_\rho [D_\rho] \in \text{Cl}(X_\Sigma). \tag{8.1}$$

If a trajectory is intended to remain in a concise technical proof class, its Cox degree should be stable across local windows. Degree drift flags tone, verbosity, or proof-mode instability.

8.2 Vector bundles of behavioral modes

A full oracle can model behavior as a vector bundle over task/context space. The base point is the problem plus context state; the fiber contains allowed behavior coordinates. Local trivializations correspond to domains: math proof, coding, medical information, creative writing, refusal, summarization, retrieval, tool use.

Definition 8.1 (Behavior vector bundle)**.** *Let X be a task-context space covered by charts U_i. A behavior vector bundle is local data $E_i = U_i \times \mathbb{R}^r$ with transition functions $g_{ij} : U_i \cap U_j \to GL_r(\mathbb{R})$ satisfying $g_{ij}g_{jk} = g_{ik}$ on triple overlaps. An oracle section is a choice of behavior vector $s_i(x) \in E_i$ compatible on overlaps.*

The bundle view prevents a common failure: using one global personality axis for all domains. Warmth, uncertainty, compliance, and verbosity mean different things in mathematical proof, emergency safety, and creative brainstorming. Transitions let the oracle adapt while preserving a global section condition.

8.3 Differential forms for drift and path dependence

Let $\xi_t \in \mathbb{R}^r$ be GraphCG behavior coordinates. A one-form $\omega = \sum_i f_i(\xi)d\xi_i$ measures local behavior work along a trajectory:

$$\int_\tau \omega = \sum_t \langle f(\xi_t), \xi_{t+1} - \xi_t \rangle. \tag{8.2}$$

If $d\omega \approx 0$, the accumulated score is approximately path-independent; if not, branch order matters. This is useful for detecting unstable tone or compliance dynamics across long reasoning paths.

8.4 Young tableaux and representation-theoretic labels

Some trajectory types are combinatorial. A proof may decompose into lemmas with dependencies; a tool plan may have interleaving resource constraints; a long answer may require ordered slots. Young tableaux provide finite normal forms for such structured labels.

Example 8.2 (Tableau label for proof obligations)**.** *Let a proof trajectory have obligations grouped by type: definitions, lemmas, computations, citations, and verifier checks. Encode the order in a semistandard tableau whose rows represent compatible parallel obligations and columns represent strict dependency. The oracle predicts the tableau shape λ, filling content, and violation count. A bad merge often appears as a column violation or a Littlewood-Richardson coefficient mismatch when combining two branches.*

Representation-theoretic labels are optional. They are most useful when a domain has genuine symmetry: root-system tasks, algebraic proof tasks, program transformations, morphology, symbolic tableaux, or structured multimodal layout. The rule remains pragmatic: use them only if they improve task quality or compression.

9 Oracle training schedule

9.1 Stage 0: audit-only

Run the base model and collect trajectory packets. Compute oracle features without training: verifier scores, property weak labels, certificate residuals, sheaf-gluing errors, retrieval quality, memory hygiene, tool-call outcomes, tone metrics, and bits-per-byte slices. This establishes label reliability and class balance.

9.2 Stage 1: frozen head

Freeze the base model. Train a linear or shallow oracle head on pooled hidden states and certificate summaries. Report per-property AUROC, calibration error, class-balanced F1, false-negative rate for high-risk properties, and mutual information with future bytes. Poor frozen-head performance indicates missing representation or poor labels.

9.3 Stage 2: adapter oracle

Unfreeze small adapters and train the head with small weights. Maintain the base language loss. Use ratio caps so auxiliary gradients cannot dominate next-token training:

$$\frac{\|\nabla \mathcal{L}_{\text{oracle}}\|}{\|\nabla \mathcal{L}_{\text{LM}}\| + \varepsilon} \leq r_{\max}. \tag{9.1}$$

9.4 Stage 3: full oracle transformer

Train Ω_ψ offline on full trajectory packets. Use it for filtering, preference labels, reward shaping, and teacher distillation. Do not deploy it unless inference-time oracle classification is a product requirement and ablations justify latency and artifact cost.

9.5 Stage 4: distillation and promotion

Distill the full oracle to a small head or quantized code. Promote only if:

1. held-out bits-per-byte improves or remains within tolerance while a predeclared reasoning/control metric improves;
2. high-risk false negatives decrease;
3. calibration remains stable out of domain;
4. artifact bytes and latency are acceptable;
5. no data-leakage or score-before-update violation is introduced.

10 Implementation blueprint

10.1 Trajectory record schema

```
@dataclass
class TrajectoryPacket:
    task_id: str
    prefix_hash: str
    step_nodes: Tensor # [T, N, d]
    graph_edges: LongTensor # graph-of-thought, retrieval, MCP, memory edges
    output_tokens: LongTensor
    verifier_scores: Tensor
    evidence_spans: list[EvidenceSpan]
    mcp_events: list[MCPEvent]
    memory_refs: list[MemoryRef]
    kg_paths: list[KGPath]
    filtered_complex: ComplexSummary
    persistence: PersistenceSummary
    derived_residuals: Tensor
    sheaf_gluing: Tensor
```

```
    toric_chart: ToricChartCode
    behavior_coordinates: Tensor
    labels: dict[str, LabelValue]
    label_sources: dict[str, list[str]]
```

10.2 Oracle head interface

```
class OracleHead(nn.Module):
    def forward(self, hidden_summary, cert_summary, context_summary):
        z = torch.cat([hidden_summary, cert_summary, context_summary], dim=-1)
        logits = self.property_logits(z)
        ordinal = self.ordinal_heads(z)
        margins = self.fan_margin_head(z)
        return OracleOutput(logits=logits, ordinal=ordinal, margins=margins)
```

10.3 Full oracle transformer interface

```
class OracleTransformer(nn.Module):
    def forward(self, packet_tokens, graph_edges, masks):
        z = self.token_embed(packet_tokens)
        z = self.graph_tropical_blocks(z, graph_edges, masks)
        pooled = invariant_pool(z, masks)
        local = self.local_property_decoder(z)
        global_out = self.global_property_decoder(pooled)
        return local, global_out
```

10.4 Minimal training loop

```
for batch in loader:
    with torch.no_grad() if freeze_base else contextlib.nullcontext():
        base_out = base_model(batch.inputs, return_trajectory=True)
    packet = build_packet(batch, base_out)
    oracle_out = oracle(packet)
    loss = supervised_loss(oracle_out, packet.labels)
    loss += lambda_poset * poset_loss(oracle_out)
    loss += lambda_sr * stanley_reisner_loss(oracle_out)
    loss += lambda_glue * sheaf_gluing_loss(oracle_out, packet.cover)
    loss += lambda_cal * calibration_loss(oracle_out, packet.labels)
    loss.backward()
    optimizer.step()
```

11 Evaluation and ablations

11.1 Oracle metrics

Report, per property and per slice:

1. AUROC, AUPRC, class-balanced F1, false-positive and false-negative rates;
2. expected calibration error and reliability plots;
3. poset violation mass and Stanley-Reisner nonface mass;

4. local-to-global gluing error;
5. fan-margin distribution and quantization stability;
6. trajectory-level agreement with human labels and verifier labels;
7. out-of-domain degradation.

11.2 Model-improvement metrics

The oracle is justified only by downstream effects:
1. held-out bits-per-byte globally and on reasoning-heavy slices;
2. verified task success and mathematical/procedural accuracy;
3. reduction in hallucinated citations and unsupported claims;
4. fewer unsafe-compliance and unnecessary-refusal cases;
5. improved retrieval precision and stale-memory rejection;
6. reduced redundant tool calls and lower context budget for same quality;
7. tone/personality corridor stability under long-context and multi-turn settings.

11.3 Ablation protocol

Ablate in this order:
1. remove oracle entirely;
2. frozen head only;
3. adapter head;
4. full oracle as data filter only;
5. full oracle as preference teacher;
6. full oracle as GFlowNet reward;
7. inference-time steering;
8. each structured loss: poset, Stanley-Reisner, sheaf gluing, toric fan, persistence, derived, calibration.

Every ablation must report artifact bytes, latency, bits-per-byte, verifier success, high-risk false negatives, and calibration.

12 Catalogue of ToricGT IV oracle objectives

The following objectives are intended as a concrete menu. Use audit-only first; train only after label reliability is acceptable; deploy only after ablation.

12.1 Classifier and calibration objectives

1. **Trajectory property cross-entropy.** Multi-label CE over atomic properties. Helps by making hidden reasoning states linearly legible for accuracy, tone, compliance, and efficiency.
2. **Ordinal severity loss.** Cumulative-link loss for severity labels: harmless issue, minor issue, major issue, catastrophic issue. Useful for safety and hallucination triage.

3. **Calibration energy.** Differentiable expected calibration error plus Brier score. Prevents overconfident steering and improves threshold reliability.
4. **Abstention/selective-risk loss.** The oracle may output "uncertain". Train coverage-risk curves so the system knows when not to trust the classifier.
5. **Property fan margin.** Encourage large margins between incompatible behavior chambers. Useful for quantized deployment and stable control.

12.2 Algebraic and topological objectives

6. **Property poset monotonicity.** Penalize entailment violations. Makes labels compositional: severe hallucination implies unsupported claim.
7. **Stanley-Reisner compatibility.** Penalize forbidden coactivations, such as high confidence and no evidence.
8. **Cech gluing loss.** Ensure local window judgments agree on overlaps. Detects local tone or evidence inconsistencies.
9. **Persistence stability loss.** Pair nearby trajectories and penalize large label changes when persistence diagrams are close.
10. **Derived residual classification.** Use mapping-cone residuals to classify branch-merge failures and invalid analogies.
11. **Cox-degree behavior drift.** Penalize unexplained changes in behavior class degree across trajectory windows.
12. **Differential-form path-dependence loss.** Penalize large $d\omega$ when behavior score should be path-independent.
13. **Vector-bundle transition loss.** Train domain-specific tone/personality transitions satisfying cocycle consistency.
14. **Tableau normal-form loss.** Classify proof or plan structures by Young-tableau shapes and penalize invalid fillings.
15. **Root-system chamber loss.** For symmetric tasks, classify Weyl chamber transitions and penalize invalid crossings.

12.3 Training-control objectives

16. **Oracle-weighted SFT.** Weight supervised tokens by oracle quality, clipped for stability.
17. **Oracle preference distillation.** Turn trajectory pairs into preference data for DPO or reward training.
18. **GFlowNet desirability reward.** Shape graph-of-thought search toward accurate, grounded, behavior-compatible trajectories.
19. **Retrieval compatibility loss.** Train memory retrieval to prefer oracle-compatible helper trajectories.
20. **MCP tool discipline loss.** Classify and penalize irrelevant, unsafe, schema-invalid, or redundant tool calls.
21. **Context economy objective.** Predict marginal utility of adding each context block; train the model to stop context filling when utility falls below cost.

22. **Bits-per-byte utility head.** Predict whether an oracle code or retrieved block improves held-out bits-per-byte after artifact cost.
23. **High-risk false-negative penalty.** Asymmetrically penalize missed unsafe compliance, hallucination, and privacy/memory leakage.
24. **Persona corridor stability.** Penalize unwarranted tone/personality drift across multi-turn or long-context trajectories.
25. **Verifier disagreement triage.** Classify cases where oracle confidence conflicts with verifier output and trigger abstention or additional evidence retrieval.

13 Pragmatic deployment rules

13.1 When to deploy a head

Deploy a small oracle head only if it satisfies all of the following:

1. high-risk properties are calibrated and false negatives are below threshold;
2. output decisions are stable under quantization and prompt paraphrase;
3. head latency is acceptable;
4. held-out bits-per-byte does not regress unless a predeclared safety/control metric improves enough to justify the cost;
5. the head does not create a future-token leakage channel;
6. the head improves at least one training or inference decision in ablation.

13.2 When to keep it teacher-side

Keep the oracle teacher-side when it improves data filtering, reward labeling, or checkpoint selection but is too expensive, too brittle, too poorly calibrated, or not useful for live inference. Most full oracle transformers will be teacher-side.

13.3 Failure modes

1. **Aesthetic overfitting:** the oracle rewards polished tone but not correctness.
2. **Sycophancy amplification:** helpfulness labels confuse agreement with usefulness.
3. **Verifier monoculture:** the oracle learns one verifier's blind spots.
4. **Label leakage:** trajectory labels accidentally encode future validation bytes.
5. **Over-steering:** inference projections degrade creativity or domain adaptation.
6. **Unsafe confidence:** calibrated properties become uncalibrated after distribution shift.
7. **Memory contamination:** oracle-compatible memories reinforce a previous wrong belief.

14 Capacity theory for oracle heads and full oracle transformers

14.1 When a head is enough

A central engineering question is whether the classifier should be a small head or a full oracle model. The answer depends on sufficiency of the base hidden representation.

Definition 14.1 (Property-sufficient representation)**.** *Let $Z_\theta(\tau)$ be a hidden representation of a trajectory packet. It is property-sufficient for labels $Y_\mathcal{P}$ if*

$$\mathbb{P}(Y_\mathcal{P} \mid \tau) = \mathbb{P}(Y_\mathcal{P} \mid Z_\theta(\tau)) \tag{14.1}$$

almost surely.

Proposition 14.2 (Frozen-head optimality under sufficiency)**.** *If Z_θ is property-sufficient and the head class contains all conditional distributions on $Y_\mathcal{P}$ as functions of Z_θ, then the optimal head attains the Bayes risk of any oracle that reads the full trajectory.*

Proof. By sufficiency, the conditional law of $Y_\mathcal{P}$ given the full packet equals the conditional law given Z_θ. Proper scoring is minimized by the true conditional law. Since the head class contains that law, it attains the full-packet Bayes risk. □

This proposition is practical: train a frozen head first. If it nearly matches a full oracle on held-out labels, the full oracle is unnecessary for deployment. If it fails specifically on tool discipline, memory freshness, or citation grounding, the base hidden state lacks those features and should be augmented or trained.

14.2 When a full oracle is necessary

The head is insufficient when property labels depend on information absent from hidden summaries: exact tool schemas, raw evidence spans, graph paths, permission boundaries, or long-context block provenance. A full oracle can recover such information by reading the trajectory packet directly.

Theorem 14.3 (Bounded-packet universal oracle approximation)**.** *Fix maximum packet size, finite type vocabularies, bounded real fields, and admissible graph relabelings. Let $f : \mathcal{T} \to \mathbb{R}^m$ be a continuous graph-invariant global property map and $g : \mathcal{T} \to \mathbb{R}^{N\times r}$ a continuous graph-equivariant local property map. A graph-token oracle transformer with equivariant tokenization, shared attention, shared feed-forward layers, and symmetric pooling can uniformly approximate (f, g) on the bounded packet space.*

Proof. The bounded packet space is a finite union of compact cells indexed by masks, graph incidence, and type patterns. On each cell, a continuous equivariant map can be conjugated through an injective equivariant tokenizer to a continuous equivariant sequence map on a compact token domain. Transformer universality for bounded sequences with shared token operations gives uniform approximation on each cell. A finite discrete selector over cells combines the approximants. Symmetric pooling gives invariant outputs, and row-wise local decoders give equivariant outputs. □

The theorem is not a claim that a finite oracle solves arbitrary-length reasoning. It says that, for the bounded packet domain used in training and validation, a full oracle transformer is expressive enough to learn the desired continuous property map if data and optimization permit.

15 A multigraded algebra of trajectory labels

15.1 Label rings

The property set has algebraic structure. Let $S_\mathcal{P} = k[z_p : p \in \mathcal{P}]$ and assign each variable a multidegree

$$\deg z_p = (d_{\text{task}}, d_{\text{evidence}}, d_{\text{behavior}}, d_{\text{safety}}, d_{\text{cost}}) \in \mathbb{Z}^5. \tag{15.1}$$

The quotient

$$R_\Delta = S_\mathcal{P}/I_\Delta \tag{15.2}$$

removes incompatible coactivations. A trajectory label distribution becomes a point in a soft version of $\operatorname{Spec} R_\Delta$. Multigrading makes it possible to ask whether the oracle is confusing evidence properties with tone properties, or cost properties with safety properties.

Definition 15.1 (Multigraded property module)**.** *A multigraded property module is a finitely generated R_Δ-module $M = \oplus_{a\in\mathbb{Z}^k} M_a$ whose graded pieces encode label families at different scopes: token, step, branch, final answer, memory event, tool event, and whole trajectory.*

A free resolution

$$\cdots \to F_2 \to F_1 \to F_0 \to M \to 0 \tag{15.3}$$

encodes relations among labels. For instance, a relation might say that "safe refusal with no alternative" and "user request answerable safely" imply an unhelpfulness warning. Syzygies among such relations capture higher-order conflicts.

15.2 Resolution-guided oracle regularization

Let $\widehat{d}_i(\tau)$ be oracle-predicted differentials between soft free modules. Define

$$\mathcal{L}_{\text{label-res}} = \sum_i \|\widehat{d}_i(\tau)\widehat{d}_{i+1}(\tau)\|_F^2 + \beta \sum_{i,a} |\widehat{\beta}_{i,a}(\tau) - \beta^\star_{i,a}|^2. \tag{15.4}$$

This is useful when label conflicts are compositional. A model that learns only independent labels can still emit impossible combinations. A resolution-aware oracle learns the relations.

Proposition 15.2 (Resolution loss detects first-order incompatibility)**.** *Suppose $F_1 \to F_0 \to M$ presents a property module and a predicted label vector violates a defining relation represented by a row of d_1. Then the corresponding component of $\widehat{d}_1\hat{z}$ is nonzero. If all first-order relations are represented in d_1, then $\widehat{d}_1\hat{z} = 0$ is exactly the linearized compatibility condition.*

Proof. A presentation identifies M with $F_0/\operatorname{im} d_1$. A vector in F_0 descends to a compatible module element exactly modulo the submodule of relations. Evaluating the relation rows on the soft vector gives zero precisely when the vector satisfies the represented linearized relations. □

16 Oracle control as a closed-loop system

16.1 Controlled dynamics

Let x_t be a reasoning state, a_t a model action such as next-token block, tool call, retrieval expansion, memory write, or branch merge, and $o_t = O_\phi(x_{0:t}, a_t)$ the oracle property vector. A controlled ToricGT process has transition kernel

$$P_\theta(dx_{t+1} \mid x_t, a_t, r_t), \tag{16.1}$$

where r_t may include retrieved memory, MCP tool output, and verifier feedback.

Definition 16.1 (Desirable invariant set)**.** *A set $K \subset \mathcal{X}$ is a desirable invariant set if states in K satisfy required bounds on accuracy risk, unsafe-compliance probability, tone/personality corridor distance, evidence support, and context cost. It is δ-invariant under policy π if*

$$\mathbb{P}_\pi(x_{t+1} \notin K \mid x_t \in K) \le \delta. \tag{16.2}$$

Theorem 16.2 (One-step risk gate)**.** *Suppose an oracle risk score $r(x, a)$ is calibrated, and action a is allowed only when $r(x, a) \le \alpha$. If the event $x_{t+1} \notin K$ is contained in the calibrated risk event, then the gated policy is α-invariant.*

Proof. For $x_t \in K$, the gate ensures $\mathbb{P}(\text{risk} \mid x_t, a_t) \le \alpha$. Since leaving K implies the risk event by assumption, $\mathbb{P}(x_{t+1} \notin K \mid x_t, a_t) \le \alpha$. Taking expectation over allowed actions preserves the bound. □

16.2 Stability versus capability

There is a tension: strong oracle gates increase reliability but may reduce capability by blocking exploratory trajectories. ToricGT IV therefore uses three levels of action:

1. **Train-time shaping:** preferred default; improves the policy before deployment.
2. **Soft inference warning:** attach additional verification or retrieval when oracle risk is high.
3. **Hard gate:** reserved for high-stakes or policy-critical actions such as unsafe tool use or memory writes.

17 Long-context oracle classification

17.1 Context-window trajectory types

In ToricGT III, the context window is a large canvas with length $L_C \leq 10^7$ filled by retrieved memory, MCP resource text, KG neighborhoods, tool outputs, scratch reasoning blocks, verifier traces, and final answer sections. ToricGT IV classifies the fill policy itself. The oracle asks whether each block has positive marginal utility, whether evidence is duplicated, whether memory is stale, whether tool returns are used correctly, and whether the final answer depends on unsupported blocks.

Definition 17.1 (Marginal context utility label)**.** *For a context block c_j inserted into prefix C, define its empirical marginal utility as*

$$U_j = \Delta\mathrm{Score}(C, c_j) - \lambda_{\mathrm{tok}}|c_j| - \lambda_{\mathrm{lat}}\mathrm{latency}(c_j) - \lambda_{\mathrm{risk}}\mathrm{risk}(c_j), \tag{17.1}$$

where ΔScore *can be verifier improvement, held-out bits-per-byte lift on a development set, retrieval recall gain, or task success improvement. The oracle label is positive when $U_j > 0$.*

17.2 Tropical ring attention as oracle evidence

Tropical ring attention exposes active predecessors and margins over long contexts. The oracle should use active evidence paths rather than all tokens uniformly. For a query block q and candidate evidence blocks e_j, define

$$A(q) = \arg\max_j \{s(q, e_j) + v(e_j)\}. \tag{17.2}$$

An answer is well grounded only if claims in the answer can be matched to active evidence blocks with stable margins. The oracle can classify low-margin claims as fragile even if the final answer appears correct.

18 Detailed use cases

18.1 Mathematical proof trajectories

A proof trajectory contains definitions, lemmas, computations, branch attempts, and final synthesis. The oracle labels missing hypotheses, invalid lemma use, circular dependency, proof gap, theorem mismatch, and overconfident final wording. Toric features enter through proof obligation complexes: definitions and lemmas are vertices, dependencies are edges, completed subproofs are higher faces, and invalid merges create boundary residuals.

18.2 Coding-agent trajectories

A coding trajectory contains repository retrieval, file edits, test runs, error traces, and patch explanations. The oracle labels whether the model used relevant files, respected constraints, ran adequate tests, avoided spurious edits, and summarized changes accurately. MCP tool-call gating is particularly useful here: classify tool schema validity and whether the next call is necessary.

18.3 Knowledge-graph question answering

A KG trajectory contains entity linking, neighborhood expansion, path selection, evidence synthesis, and answer generation. The oracle labels path relevance, unsupported relation jumps, stale KG edges, citation mismatch, and path redundancy. Toric property chambers separate "enough evidence", "needs disambiguation", and "unsupported answer".

18.4 Markdown memory and persona stability

A markdown memory trajectory contains read, select, update, and write operations over memory blocks. The oracle labels whether a memory is relevant, stale, user-specific, privacy-sensitive, or behaviorally contaminating. Personality control is handled as a bundle: the same style vector has different permitted intervals depending on task chart.

18.5 Safety and refusal trajectories

A safety trajectory contains intent classification, policy-boundary reasoning, allowed-content extraction, refusal text, and safe alternative. The oracle labels unnecessary refusal, unsafe compliance, missing safe alternative, excessive moralizing, and tone mismatch. Stanley-Reisner nonfaces prevent incompatible labels such as "unsafe compliance" and "safe refusal" from both being high.

18.6 Scientific literature synthesis

A literature trajectory contains retrieval, relevance screening, claim extraction, contradiction tracking, and final synthesis. The oracle labels citation support, overgeneralization, uncertainty quality, and whether the output distinguishes evidence from speculation. Differential-form path-dependence can flag cases where conclusion changes depending on evidence order.

18.7 Long-context summarization

A summarization trajectory fills a large context with sections and decides what to keep. The oracle labels coverage, redundancy, hallucinated compression, omission of critical caveats, and context economy. Marginal context utility labels train the model to stop adding low-value blocks.

18.8 Tool-augmented data analysis

A data-analysis trajectory contains table loading, transformation, plotting, statistical testing, and final interpretation. The oracle labels schema mismatch, invalid aggregation, overclaiming, missing uncertainty, and plot-text inconsistency.

18.9 Multi-agent debate and verifier loops

A debate trajectory contains multiple candidate solutions, critiques, verifier checks, and final aggregation. The oracle labels whether critique actually addresses evidence, whether final aggregation resolves contradictions, and whether one branch dominates without justification.

18.10 Behavior-preserving translation or rewriting

A rewriting trajectory must preserve meaning while changing tone, register, or language. The oracle labels semantic drift, tone success, personality compatibility, and unnecessary content insertion. Representation-theoretic normal forms can help when transformations have compositional structure.

19 Extended catalogue of oracle-derived objectives

19.1 Data and label objectives

4. **Multi-source denoising objective.** Estimate source reliability and train on posterior soft labels rather than raw labels.
5. **Counterfactual edit objective.** Edit one property of a trajectory while preserving all others; penalize unintended label changes.
6. **Slice-balanced property loss.** Equalize performance across math, coding, safety, retrieval, and natural prose slices.
7. **Adversarial label stress test.** Generate near-boundary trajectories and require calibrated uncertainty.
8. **Human-oracle disagreement mining.** Sample high-disagreement cases for human review.

19.2 Control and deployment objectives

9. **Safe-action threshold loss.** Optimize calibrated thresholds for MCP calls, memory writes, and high-risk answers.
10. **Oracle-triggered retrieval loss.** When the oracle predicts low grounding, train the agent to retrieve more evidence rather than hallucinate.
11. **Oracle-triggered clarification loss.** When intent ambiguity is high, train the model to ask a concise clarifying question.
12. **Tone repair objective.** Generate minimal edits that move a response into the tone corridor without changing factual content.
13. **Memory quarantine objective.** Learn to mark memories that are stale, untrusted, or behaviorally contaminating.
14. **Tool-call economy objective.** Penalize tool calls whose oracle-predicted marginal utility is below cost.
15. **Verifier escalation objective.** Trigger external verification when oracle confidence and task risk cross a threshold.
16. **Long-context pruning objective.** Drop context blocks with negative predicted marginal utility.
17. **Oracle consistency under paraphrase.** Require stable labels under input paraphrases and graph isomorphisms.
18. **Quantized-head stability objective.** Train logits so int8 or low-rank quantization preserves property chambers.

19.3 Geometric and algebraic objectives

19. **Property polytope occupancy entropy.** Avoid collapse to one behavior chamber except where appropriate.
20. **One-dimensional-cone axis sparsity.** Encourage sparse use of primitive behavior axes for interpretability.
21. **Cech disagreement localization.** Attribute global label failure to the smallest local window causing overlap mismatch.
22. **Vector-bundle cocycle audit.** Ensure behavior-coordinate transitions compose consistently across task charts.
23. **Derived cone failure classifier.** Predict whether a branch merge fails because of missing evidence, conflicting assumptions, or invalid transformation.
24. **Tableau branch-order loss.** Penalize proof or plan branch order that violates semistandard constraints.
25. **Weyl-chamber symmetry loss.** Require symmetric tasks to receive labels transformed by the corresponding group action.
26. **Ergodic behavior budget loss.** Penalize long-run frequency of undesirable trajectory types above a bound.
27. **Oracle mutual-information estimator.** Estimate $I(B; Z \mid C)$ for oracle codes and prune low-utility codes.
28. **Artifact-aware Pareto objective.** Optimize quality, risk, latency, artifact bytes, and bits-per-byte as a Pareto frontier rather than a single scalar.

20 A complete ToricGT IV training recipe

20.1 Recommended default

The default recipe is deliberately conservative.

1. Collect trajectories from the base model, including failed and low-quality samples.
2. Compute finite ToricGT certificates and ordinary verifier metrics.
3. Create a small property taxonomy with no more than 20 initial labels.
4. Train a frozen oracle head and audit calibration.
5. Use the oracle only for data selection and trajectory rejection.
6. Train a base-model ablation on the filtered data.
7. If verified quality improves without bits-per-byte regression, add preference pairs.
8. If preference training succeeds, train a full oracle teacher for difficult labels.
9. Distill only the useful subset back into a compact head.

20.2 Escalation path

Escalate to a full oracle transformer only when at least one of the following holds: the head misses labels requiring raw evidence spans; graph-of-thought topology matters; long-context block attribution matters; MCP tool schemas matter; or local-to-global behavior gluing cannot be recovered from pooled hidden states.

21 Limits and safety considerations

21.1 Oracle labels are not ground truth

An oracle can be wrong, biased, overfit, or miscalibrated. Its outputs should be treated as probabilistic training signals. For high-stakes actions, oracle labels should trigger additional verification rather than automatic suppression or compliance.

21.2 Do not optimize personality at the expense of truth

Tone and personality are secondary to factuality, safety, and user intent. The objective weights should reflect this. A polite hallucination is worse than a terse correct answer when accuracy is central.

21.3 Avoid hidden chain exposure

The oracle can operate on hidden summaries, graph certificates, and local labels without exposing private reasoning text. Training should not require showing full hidden chain-of-thought to users. Instead, expose concise rationales, citations, verifier status, and uncertainty when appropriate.

22 Discussion

ToricGT IV reframes behavior control as trajectory typing. This is stricter than final-answer classification and more pragmatic than unconstrained interpretability. The oracle does not need to expose private chain-of-thought text; it can classify finite trajectory packets, hidden summaries, graph structure, evidence events, and certificate residuals. The mathematical role of toric geometry, sheaves, persistence, representation theory, and ergodic dynamics is to make those labels structured, compositional, auditable, and compact.

The most important engineering distinction is between *oracle accuracy* and *oracle utility*. A full oracle transformer might classify trajectories beautifully and still be useless for deployment if it adds latency, artifact bytes, or false security. Conversely, a small head with only ten well-calibrated properties may produce substantial gains if it filters bad training data, improves GFlowNet search, rejects stale memory, or catches high-risk false negatives. The research program should therefore report both scientific oracle metrics and downstream model-improvement metrics.

23 Conclusion

ToricGT IV completes a natural arc. ToricGT I supplies finite algebraic certificates for graph-token reasoning. ToricGT II studies their dynamics through persistence, sheaves, vector bundles, differential forms, derived objects, and ergodic summaries. ToricGT III moves them into external context systems: MCPs, markdown memory, knowledge graphs, vector retrieval, quantization, and long-context tropical ring attention. ToricGT IV trains an oracle to classify the resulting trajectories by properties that matter: accuracy, reasoning quality, compliance, safety, tone, personality, efficiency, retrieval discipline, memory hygiene, and bits-per-byte utility.

The oracle is useful when it changes training decisions. It can select better data, shape graph-of-thought search, train preferences, control retrieval, gate MCP calls, prevent stale-memory use, improve calibration, and distill compact control signals into the base model. Its mathematical structure is not optional ornamentation; it is what makes the classifier compositional and auditable. But the final criterion remains empirical and severe: retain only the pieces that improve held-out bits-per-byte, verified reasoning, behavior reliability, or artifact efficiency.

A Additional proof details

A.1 Stanley-Reisner gradients

For a minimal nonface F, the penalty $\prod_{p\in F}\hat{y}_p$ has gradient

$$\frac{\partial}{\partial \hat{y}_q}\prod_{p\in F}\hat{y}_p = \begin{cases}\prod_{p\in F\setminus\{q\}}\hat{y}_p, & q\in F,\\ 0, & q\notin F.\end{cases} \tag{A.1}$$

Thus the penalty concentrates on jointly high forbidden activations and is small when at least one incompatible property is confidently low.

A.2 A conservative threshold rule

Let $p_{\text{risk}}(\tau)$ be a calibrated risk probability and $C_{\text{FN}}, C_{\text{FP}}$ be false-negative and false-positive costs. The Bayes action is to block or escalate when

$$p_{\text{risk}}(\tau)C_{\text{FN}} > (1-p_{\text{risk}}(\tau))C_{\text{FP}}. \tag{A.2}$$

Equivalently, block when $p_{\text{risk}}(\tau) > C_{\text{FP}}/(C_{\text{FN}}+C_{\text{FP}})$. This is why calibration is not cosmetic: it directly determines threshold correctness.

B Recommended initial experiment

A minimal ToricGT IV experiment should avoid the full oracle at first.

1. Generate 50,000 trajectory packets from existing ToricGT checkpoints on math, coding, tool-use, retrieval, and safety-style tasks.
2. Label 12 properties: correctness, verifier agreement, grounding, hallucination risk, proof rigor, tool relevance, memory freshness, safe refusal, unsafe compliance, tone match, verbosity match, and context efficiency.
3. Train a frozen oracle head on pooled hidden/certificate summaries.
4. Evaluate calibration and slice performance.
5. Use the head for data filtering only; train a base-model SFT ablation.
6. If bits-per-byte and verified quality improve, add preference distillation and GFlowNet reward shaping.
7. Train a full oracle transformer only after the head proves that labels are useful.

References

[1] David Cox, John Little, and Hal Schenck. *Toric Varieties*. American Mathematical Society, 2011.

[2] Amelie Schreiber. *ToricGT with Toric BGG Supervision: Graph-Token Reasoning, Hypertoric Category O, and bits-per-byte-constrained Training*. 2026.

[3] Amelie Schreiber. *ToricGT II: Dynamical Persistence, Ergodic Toric Geometry, and Derived-Categorical Training Objectives for Reasoning Agents*. 2026.

[4] Amelie Schreiber. *ToricGT III: Sheafified MCP Memory, Graph-Structured Vector Retrieval, and Long-Context Tropical Ring Attention*. 2026.

[5] Model Context Protocol. *Specification 2025-11-25*. 2025.

[6] I. N. Bernstein, I. M. Gelfand, and S. I. Gelfand. Differential operators on the base affine space and a study of $\mathfrak{g}$-modules. 1970s.

[7] A. Klyachko. Equivariant bundles on toral varieties. 1989.

[8] Sergei Gelfand and Yuri Manin. *Methods of Homological Algebra*. Springer, 2003.

[9] Herbert Edelsbrunner and John Harer. *Computational Topology: An Introduction*. AMS, 2010.

[10] Robin Hartshorne. *Algebraic Geometry*. Springer, 1977.

[11] William Fulton. *Introduction to Toric Varieties*. Princeton University Press, 1993.

[12] Richard Stanley. *Combinatorics and Commutative Algebra*. Birkhauser, 1996.